

Titre du cours : Mise en place d'un
écosystème d'IA

Code officiel : 420-A57-BB

Professeur : Dr Komi SODOKE

MODULE 02

Mettre en place un environnement matériel dans le contexte de l'IA

Spécialisation technique en Intelligence Artificielle

Systemes & Pilotes

Langages & IDE

Frameworks ML

PLAN DU COURS

Objectif terminal : Mettre en place un environnement matériel dans le contexte de l'IA

I

Introduction

Pourquoi la mise en place de l'environnement est cruciale pour l'IA

II

Environnements matériels

Systèmes d'exploitation · Installation OS & Pilotes · Mises à jour · Réseautique

III

Environnements matériels et logiciels

Langages de programmation · IDE · Frameworks ML (PyTorch, TensorFlow, Keras...)

IV

Questions de compréhension

Quiz interactif pour valider les acquis du module

V

Conclusion & Synthèse

Récapitulatif et validation de l'environnement complet

I

Introduction

Pourquoi la mise en place de l'environnement est cruciale pour tout projet d'IA



Avoir du bon matériel ne suffit pas !

Dans le Module 1, nous avons appris à choisir les bonnes composantes (CPU, GPU, RAM...). Mais ces composantes sont inutiles sans une configuration logicielle adaptée. C'est comme acheter une voiture de course sans mettre d'essence ni régler le moteur.



Le matériel

= *La voiture*

CPU, GPU, RAM, SSD... Ce sont les composantes physiques que nous avons sélectionnées dans le Module 1.



Le système d'exploitation

= *L'essence + la route*

Linux ou Windows : c'est le logiciel de base qui permet au matériel de fonctionner et d'exécuter des programmes.



Les frameworks IA

= *Le pilote expert*

PyTorch, TensorFlow : ce sont les outils qui permettent de créer, entraîner et déployer des modèles d'intelligence artificielle.

I – Un GPU puissant ne suffit pas

Section I



Exemple concret : Vous avez une NVIDIA RTX 4090 (24 Go de VRAM). Vous lancez un entraînement PyTorch...

Résultat : PyTorch utilise le CPU au lieu du GPU. L'entraînement qui devrait prendre 2 heures prend 2 semaines. Pourquoi ? Parce que les drivers NVIDIA, CUDA et cuDNN n'ont pas été installés correctement.



Drivers absents

Sans le pilote NVIDIA, le système ne détecte même pas la carte graphique. Le GPU reste invisible.



CUDA non configuré

CUDA est la plateforme NVIDIA qui permet d'utiliser le GPU pour le calcul parallèle. Sans CUDA, pas de Deep Learning sur GPU.



OS non optimisé

Un système non mis à jour peut avoir des conflits de noyau (kernel) qui empêchent la communication avec le GPU.



Réseau mal configuré

Impossible de télécharger les datasets, d'accéder au cloud ou de faire de l'entraînement distribué sur plusieurs machines.

I – Objectifs du module & compétences visées

Section I

À l'issue de ce module, vous serez capable de :



Installer

Installer un système d'exploitation (Linux Ubuntu ou Windows) optimisé pour l'IA sur une machine de calcul.



Configurer

Installer et configurer les pilotes GPU (NVIDIA), CUDA et cuDNN pour activer l'accélération matérielle.



Mettre à jour

Appliquer les mises à jour système et pilotes de manière sécurisée, sans casser l'environnement existant.



Réseauter

Configurer le réseau (IP, DNS, SSH) pour permettre l'accès distant et le téléchargement de données massives.



Outils

Installer les langages (Python, C++), les IDE (VS Code, Jupyter) et les frameworks ML (PyTorch, TensorFlow).



Valider

Vérifier que l'ensemble de l'écosystème fonctionne correctement via des tests de validation (nvidia-smi, torch.cuda).



Environnements matériels

Systèmes d'exploitation · Installation · Mises à jour · Réseautique



II.A — Qu'est-ce qu'un système d'exploitation ?

Section II



Définition

Un système d'exploitation (OS — Operating System) est un logiciel fondamental qui sert d'intermédiaire entre l'utilisateur et le matériel informatique. Il gère le processeur, la mémoire, le stockage, les périphériques et permet d'exécuter des applications.



Analogie : L'OS est le « chef d'orchestre » de votre ordinateur. Sans lui, le processeur, la RAM et le GPU ne savent pas quoi faire — comme des musiciens sans partition.

Les trois rôles principaux d'un OS :

Gestion du matériel

Alloue le CPU, la RAM et le GPU aux programmes qui en ont besoin.

Interface utilisateur

Permet d'interagir avec la machine (terminal, interface graphique).

Exécution des logiciels

Lance et supervise les applications (Python, PyTorch, navigateur...).

II.A — Linux vs Windows : quel OS pour l'IA ?

Section II



Linux (Ubuntu)

Le roi de l'IA

- Gratuit et open-source : pas de licence à payer
- 95% des serveurs d'IA tournent sous Linux
- Compatible nativement avec CUDA, PyTorch, TensorFlow
- Contrôle total du système (noyau, pilotes, services)
- Gestion efficace de la mémoire et des processus
- Ubuntu 22.04 LTS : version recommandée pour l'IA



Windows

L'alternative accessible

- Interface graphique familière pour les débutants
- Bon support NVIDIA avec pilotes Windows
- WSL2 : exécuter un Linux complet dans Windows
- Idéal pour le développement local et le prototypage
- Moins optimisé pour les serveurs de production IA
- Licence payante (Windows 10/11 Pro recommandé)

II.A — Linux en détail : distributions et choix

Section II

Une « distribution » Linux est un OS complet basé sur le noyau Linux, avec ses propres logiciels et sa propre interface.

Distribution	Utilisation en IA	Avantages	Niveau
Ubuntu 22.04 LTS	La référence absolue	Immense communauté, docs NVIDIA officielles, compatible tout framework	Débutant
Debian	Serveurs stables	Ultra-stable, base d'Ubuntu, parfait pour la production long terme	Intermédiaire
CentOS / Rocky	Entreprise / HPC	Compatible Red Hat, utilisé dans les data centers professionnels	Avancé



Recommandation pour ce cours

Nous utiliserons Ubuntu 22.04 LTS (Long Term Support). « LTS » signifie que cette version reçoit des mises à jour de sécurité pendant 5 ans, ce qui la rend idéale pour un environnement de production IA stable. C'est aussi la distribution pour laquelle NVIDIA fournit des guides d'installation officiels de CUDA.

II.A — Windows + WSL2 : le meilleur des deux mondes

Section II



WSL2 (Windows Subsystem for Linux 2) permet de faire tourner un vrai noyau Linux à l'intérieur de Windows, avec accès au GPU NVIDIA. Idéal pour les étudiants qui veulent garder Windows tout en développant dans un environnement Linux.

Comment ça fonctionne ?

- 1 Windows reste votre OS principal (interface graphique, navigateur, applications).
- 2 WSL2 crée une machine virtuelle Linux légère qui partage les ressources de Windows.
- 3 Vous accédez à un terminal Ubuntu complet depuis Windows (bash, apt, pip, conda).
- 4 Le GPU est partagé : PyTorch dans WSL2 peut utiliser votre carte NVIDIA Windows.

```
Installation : wsl --install -d Ubuntu-22.04
```

II.A — En entreprise : quel OS pour quel usage ?

Section II

Contexte	OS recommandé	Pourquoi ?
Serveur d'entraînement IA	Linux (Ubuntu Server)	Performance maximale, pas d'interface graphique inutile, stabilité 24/7
Poste développeur local	Windows + WSL2 ou Ubuntu Desktop	Confort d'utilisation + accès aux outils IA via terminal Linux
Cluster GPU (HPC)	Linux (CentOS / Rocky)	Standard dans les data centers, compatible avec les outils de cluster (SLURM, MPI)
Déploiement cloud (AWS, GCP)	Linux (Ubuntu / Amazon Linux)	Images pré-configurées avec CUDA et Docker disponibles sur toutes les plateformes cloud

À retenir

Linux domine largement le monde de l'IA professionnelle. Même si vous développez sous Windows, votre modèle sera presque toujours déployé sur un serveur Linux. Maîtriser Linux n'est pas optionnel — c'est une compétence fondamentale en IA.

II.B – Installer le système d'exploitation

Section II

L'installation d'un OS est la première étape concrète. Voici le processus complet pour installer Ubuntu sur une machine dédiée à l'IA.

1

Télécharger l'image ISO

Rendez-vous sur ubuntu.com et téléchargez Ubuntu 22.04 LTS (Server ou Desktop selon le besoin).

2

Créer une clé USB bootable

Utilisez Rufus (Windows) ou dd (Linux) pour graver l'ISO sur une clé USB d'au moins 8 Go.

3

Configurer le BIOS/UEFI

Démarrez sur la clé USB en modifiant l'ordre de boot dans le BIOS. Désactivez le Secure Boot si nécessaire.

4

Partitionner le disque

Divisez le disque : partition système (200-300 Go), partition données/datasets (reste de l'espace), swap (= RAM).

5

Installer et redémarrer

Suivez l'assistant d'installation, choisissez le fuseau horaire, créez un utilisateur, puis redémarrez sans la clé USB.

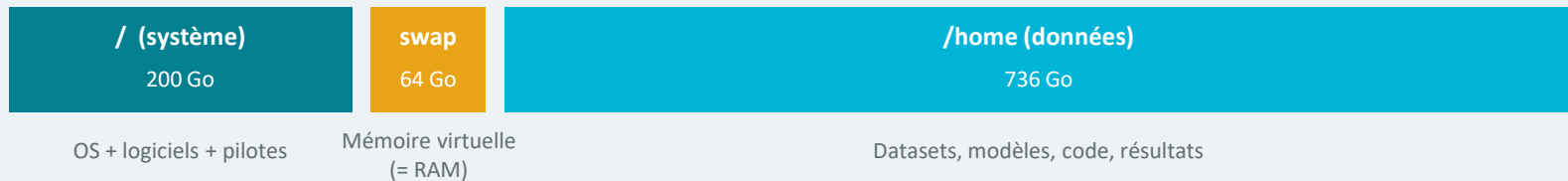
II.B — Focus : le partitionnement pour l'IA

Section II



Partitionnement : diviser un disque dur en sections indépendantes. Chaque partition se comporte comme un disque séparé. Le formatage prépare la partition à recevoir des fichiers (format ext4 pour Linux, NTFS pour Windows).

Exemple : Machine IA avec SSD 1 To



Pourquoi séparer système et données ?

- Si le système plante, vos datasets et modèles sont protégés sur une partition séparée.
- Vous pouvez réinstaller l'OS sans perdre vos données d'entraînement.
- Le swap (mémoire virtuelle) évite les crashes quand la RAM est saturée lors d'un entraînement lourd.

II.B — Les pilotes (Drivers) : définition et rôle

Section II



Définition

Un pilote (driver) est un petit logiciel qui sert de traducteur entre le système d'exploitation et un composant matériel. Sans le bon pilote, le système ne sait pas comment « parler » au matériel — le composant reste invisible et inutilisable.

Types de pilotes importants en IA :



Pilotes GPU (NVIDIA)

CRITIQUE

Le plus critique en IA. Permet au système de détecter et d'utiliser la carte graphique pour le calcul parallèle.



Pilotes réseau

IMPORTANT

Permettent la communication réseau (Ethernet, Wi-Fi). Essentiels pour télécharger des datasets et accéder au cloud.



Pilotes stockage

IMPORTANT

Gèrent les disques NVMe et contrôleurs RAID. Impactent la vitesse de lecture des datasets.

II.B — La chaîne GPU : Driver → CUDA → cuDNN

Section II

Pour utiliser un GPU NVIDIA en Deep Learning, trois couches logicielles doivent être installées dans l'ordre. Chacune dépend de la précédente.

1

Driver NVIDIA

= La clé qui démarre le moteur

Le pilote de base. Permet au système d'exploitation de détecter la carte graphique et de communiquer avec elle. Commande de vérification :

```
$ nvidia-smi
```

2

CUDA Toolkit

= Le moteur qui fait avancer la voiture

Compute Unified Device Architecture. Plateforme de calcul parallèle NVIDIA qui permet d'exécuter des milliers de calculs simultanés sur les cœurs du GPU.

```
$ nvcc --version
```

3

cuDNN

= Le turbo qui accélère le moteur

CUDA Deep Neural Network library. Bibliothèque optimisée pour les opérations de réseaux de neurones (convolutions, RNN, normalisation).

```
$ cat /usr/include/cudnn_version.h
```

II.B — Installation pratique des pilotes GPU (Ubuntu)

Section II

Voici les commandes à exécuter dans l'ordre pour installer la pile GPU complète sur Ubuntu 22.04 :

Étape 1 — Mettre à jour le système

```
$ sudo apt update && sudo apt upgrade -y
```

Toujours commencer par mettre à jour avant d'installer quoi que ce soit.

Étape 2 — Installer le driver NVIDIA

```
$ sudo apt install nvidia-driver-535
```

Le numéro (535) correspond à la version. Vérifiez la plus récente compatible.

Étape 3 — Redémarrer

```
$ sudo reboot
```

Le driver n'est actif qu'après un redémarrage complet du système.

Étape 4 — Vérifier le driver

```
$ nvidia-smi
```

Doit afficher le nom du GPU, la VRAM et la version du driver.

Étape 5 — Installer CUDA Toolkit

```
$ sudo apt install nvidia-cuda-toolkit
```

Installe le compilateur nvcc et les bibliothèques CUDA.

Étape 6 — Installer cuDNN

```
$ # Télécharger depuis developer.nvidia.com
```

Nécessite un compte NVIDIA Developer (gratuit).

II.C – Pourquoi mettre à jour le système ?

Section II

Les mises à jour ne sont pas optionnelles. Elles jouent un rôle critique dans la stabilité et la sécurité de votre environnement IA.



Sécurité

Les vulnérabilités sont corrigées. Un serveur IA non mis à jour est une cible pour les cyberattaques (vol de modèles, crypto-mining).



Correction de bugs

Les mises à jour corrigent des erreurs dans le noyau Linux, les pilotes et les bibliothèques qui peuvent causer des crashes.



Performances

Les nouvelles versions de pilotes NVIDIA apportent souvent des gains de performance significatifs (jusqu'à 10-15% par génération).



Compatibilité

Les frameworks IA (PyTorch, TensorFlow) exigent souvent des versions minimales de CUDA et du driver. Ne pas mettre à jour = ne pas pouvoir installer les dernières versions.

II.C – Commandes de mise à jour et bonnes pratiques

Section II

Commandes essentielles (Ubuntu/Debian)

```
$ sudo apt update
```

Met à jour la liste des paquets disponibles (ne modifie rien sur le système).

```
$ sudo apt upgrade -y
```

Installe les mises à jour disponibles pour tous les paquets existants.

```
$ sudo apt dist-upgrade
```

Gère les changements de dépendances (ajout/suppression de paquets si nécessaire).

```
$ sudo apt autoremove
```

Supprime les paquets obsolètes qui ne sont plus nécessaires.



Attention : mise à jour des pilotes GPU

Ne mettez JAMAIS à jour le driver NVIDIA en plein entraînement ! Sauvegardez votre travail, arrêtez les processus GPU, puis mettez à jour et redémarrez. Une mise à jour de driver interrompue peut rendre le GPU inutilisable.

Bonnes pratiques :

- Planifier les mises à jour pendant les périodes sans entraînement actif.
- Tester sur une machine de développement avant d'appliquer sur le serveur de production.
- Garder une copie de l'ancien driver fonctionnel au cas où le nouveau pose problème.

II.D – Configuration réseau : concepts essentiels

Section II

La réseautique (networking) est la configuration qui permet à votre machine IA de communiquer avec d'autres machines, d'accéder à Internet et de transférer des données.



Adresse IP

Identifiant unique de votre machine sur le réseau. Exemple : 192.168.1.100. Comme une adresse postale, elle permet aux autres machines de vous trouver.



DNS

Domain Name System. Traduit les noms de domaine (ex : google.com) en adresses IP. Sans DNS, vous devriez mémoriser des séries de chiffres pour naviguer.



Passerelle (Gateway)

La « porte de sortie » vers Internet. C'est généralement l'adresse de votre routeur (ex : 192.168.1.1). Sans passerelle, votre machine ne peut pas accéder au monde extérieur.



Bande passante

Vitesse de transfert des données sur le réseau (ex : 1 Gbps = 125 Mo/s). Un réseau lent ralentit le téléchargement des datasets et l'entraînement distribué.



SSH (Secure Shell)

Protocole permettant de se connecter à distance à une machine Linux de manière sécurisée. Indispensable pour administrer des serveurs IA sans écran.



Téléchargement de datasets

Les jeux de données en IA sont massifs. ImageNet pèse 150 Go, Common Crawl dépasse 80 To. Un réseau à 100 Mbps prend 3h30 pour 150 Go. Un réseau à 1 Gbps : 20 minutes. Le choix de la bande passante impacte directement votre productivité.



Accès cloud & SSH

Souvent, on développe localement mais on entraîne sur un serveur distant (AWS, GCP) via SSH. La configuration réseau doit permettre un accès stable, avec une IP fixe ou un bail DHCP réservé pour ne pas perdre la connexion en plein entraînement.



Entraînement distribué

Les très grands modèles (LLM) nécessitent plusieurs GPU sur plusieurs machines. Ces machines communiquent en permanence via le réseau (NCCL, MPI). Un réseau lent crée un goulot d'étranglement qui annule le bénéfice des GPU supplémentaires.

II.D — Configuration réseau pratique (Ubuntu)

Section II

Ubuntu utilise Netplan pour la configuration réseau. Voici comment configurer une IP statique sur votre machine IA :

Fichier : `/etc/netplan/01-config.yaml`

```
network:
  version: 2
  ethernets:
    ens33:
      addresses:
        - 192.168.1.100/24
      gateway4: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
```

ens33

Nom de l'interface réseau (carte Ethernet)

addresses

IP fixe de la machine (192.168.1.100)

/24

Masque de sous-réseau (255.255.255.0)

gateway4

Adresse du routeur (passerelle)

nameservers

Serveurs DNS (ici Google: 8.8.8.8)

Commandes pour appliquer :

```
$ sudo netplan apply
$ ip addr show      # Vérifier l'IP
$ ping 8.8.8.8      # Tester la connexion
```

Activer SSH :

```
sudo apt install openssh-server
sudo systemctl enable ssh
```



Environnements matériels et logiciels

Langages · IDE · Frameworks d'apprentissage machine



III.A — Langages de programmation pour l'IA

Section III

L'IA utilise plusieurs langages, chacun ayant un rôle spécifique. Voici les principaux et pourquoi ils sont utilisés.

Langage	Rôle en IA	Pourquoi ?	Usage
Python	Le leader incontesté	Syntaxe simple, écosystème immense (NumPy, Pandas, Scikit-learn, PyTorch, TensorFlow)	90%+
C++	La performance brute	Moteur de calcul sous-jacent des frameworks (TensorFlow écrit en C++)	~5%
R	La statistique	Très puissant pour l'analyse de données et les visualisations. Utilisé en recherche.	~3%
Matlab	L'ingénierie	Traitement du signal, automatique, simulations. Payant (licence universitaire souvent).	~2%



Python : pourquoi est-il n°1 en IA ?

Python est simple à apprendre (syntaxe lisible comme de l'anglais), dispose de milliers de bibliothèques scientifiques, et est le langage natif de tous les grands frameworks IA (PyTorch, TensorFlow, Keras, Scikit-learn). Quand NVIDIA optimise CUDA pour l'IA, c'est l'interface Python qui est prioritaire. Conclusion : maîtriser Python est obligatoire pour tout praticien de l'IA.

III.A — Python en détail : installation et bibliothèques

Section III

Installation de Python pour l'IA :

```
$ sudo apt install python3 python3-pip python3-venv
$ python3 --version      # Vérifier : Python 3.10+
$ pip3 install --upgrade pip
```



Bonne pratique : toujours utiliser un environnement virtuel (venv) pour isoler les dépendances de chaque projet. Un projet peut nécessiter PyTorch 2.0 et un autre PyTorch 1.13 — le venv évite les conflits.

Bibliothèques Python essentielles en IA :

Bibliothèque	Rôle	Commande d'installation
NumPy	Calcul matriciel (base de tout)	<code>pip install numpy</code>
Pandas	Manipulation de données tabulaires	<code>pip install pandas</code>
Matplotlib	Visualisation de données (graphiques)	<code>pip install matplotlib</code>
Scikit-learn	ML classique (classification, régression)	<code>pip install scikit-learn</code>

III.B — Environnement de développement (IDE)

Section III



Définition

Un IDE (Integrated Development Environment) est un logiciel qui regroupe tous les outils nécessaires pour écrire, tester et déboguer du code : éditeur de texte intelligent, terminal intégré, débogueur, et autocomplétion. C'est votre « atelier de travail » en tant que développeur IA.

VS Code

RECOMMANDÉ

Le plus polyvalent

Gratuit, léger, avec des extensions Python et Jupyter. Supporte SSH pour coder à distance sur un serveur GPU. Le choix n°1 des développeurs IA.

PyCharm

PRO

Le plus professionnel

IDE dédié à Python par JetBrains. Très puissant pour les gros projets : refactoring, tests automatisés, intégration Docker. Version gratuite (Community) disponible.

Jupyter Notebook

ESSENTIEL

Le favori des Data Scientists

Permet de mélanger code, texte et graphiques dans un « notebook » interactif. Parfait pour l'exploration de données et le prototypage. Fonctionne dans le navigateur.

III.B — Focus : Jupyter Notebook

Section III

Jupyter est un outil incontournable en IA. Voici pourquoi et comment l'utiliser.

```
$ pip install jupyter notebook
$ jupyter notebook          # Lance l'interface dans le navigateur
```

Ce qui rend Jupyter unique :

- ✓ **Cellules de code** Exécutez le code par morceaux (cellules). Idéal pour tester étape par étape un pipeline de données ou un modèle.
- ✓ **Cellules Markdown** Mélangez du texte explicatif, des formules LaTeX et des images avec votre code. Parfait pour documenter vos expériences.
- ✓ **Visualisation inline** Les graphiques Matplotlib, les tableaux Pandas et les images s'affichent directement sous la cellule de code.
- ✓ **Accès distant** JupyterLab peut tourner sur un serveur GPU et être accessible via navigateur depuis n'importe quel poste.

JupyterLab : la version évoluée de Jupyter Notebook. Interface plus riche avec explorateur de fichiers, terminal intégré, et multi-onglets.
Installation : `pip install jupyterlab`. Recommandé pour les projets sérieux.

III.C — Qu'est-ce qu'un framework d'apprentissage machine ?

Section III



Définition

Un framework d'apprentissage machine est une boîte à outils logicielle pré-construite qui fournit les briques de base pour créer, entraîner et déployer des modèles d'IA. Il contient les fonctions mathématiques (calcul tensoriel, rétropropagation, optimiseurs) pour que vous n'ayez pas à tout recoder depuis zéro.



Analogie : Un framework est comme une cuisine professionnelle équipée. Plutôt que de forger vos couteaux et construire votre four, vous utilisez des outils professionnels prêts à l'emploi pour vous concentrer sur la recette (votre modèle IA).

Ce qu'un framework fournit :



Tenseurs & GPU

Structures de données optimisées (tenseurs) qui tournent sur GPU via CUDA automatiquement.



Couches de neurones

Couches pré-construites (Dense, Conv2D, LSTM, Transformer) prêtes à assembler.



Rétropropagation auto

Calcul automatique des gradients (autograd) — le cœur de l'apprentissage.



Optimiseurs

Algorithmes d'optimisation (SGD, Adam, AdamW) pour ajuster les poids du modèle.

PyTorch

Le framework préféré des chercheurs et de l'industrie

- Graphe dynamique : le réseau se construit à la volée pendant l'exécution. Le débogage se fait comme du Python normal (print, breakpoint).
- Autograd : calcul automatique des gradients. Chaque opération est enregistrée et la rétropropagation se fait en une ligne (loss.backward()).
- Écosystème riche : torchvision (images), torchaudio (sons), torchtext (texte), HuggingFace Transformers utilise PyTorch par défaut.
- GPU natif : passer un modèle sur GPU est simple : model.to('cuda'). Les tenseurs se déplacent entre CPU et GPU facilement.
- Communauté immense : la majorité des papiers de recherche publient leur code en PyTorch. C'est le standard de facto en 2024+.

```
$ pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
```

cu121 = version CUDA 12.1 — adapter selon votre version de CUDA installée

TensorFlow

Le framework de production de Google

- Développé par Google Brain, très robuste pour la mise en production.
- TF Serving : déploiement de modèles à grande échelle sur serveurs.
- TF Lite : modèles optimisés pour mobile et embarqué (smartphones, IoT).
- Graphe statique (compilé) : plus rapide en production mais moins intuitif pour le développement.
- Écosystème complet : TensorBoard pour la visualisation des entraînements.

Keras

L'interface simplifiée de TensorFlow

- API de haut niveau intégrée dans TensorFlow (tf.keras).
- Syntaxe ultra-simple : construire un réseau en quelques lignes.
- Idéal pour l'apprentissage et le prototypage rapide.
- `model.fit()` : une seule commande pour entraîner un modèle complet.
- Depuis Keras 3.0 : compatible avec PyTorch, TensorFlow ET JAX.

```
$ pip install tensorflow
```

```
# Inclut Keras automatiquement
```

III.C – Comparaison des frameworks : lequel choisir ?

Section III

Critère	PyTorch	TensorFlow	Keras	Caffe
Créateur	Meta (Facebook)	Google	Google (F. Chollet)	UC Berkeley
Facilité	★★★★☆	★★★☆☆	★★★★★	★★☆☆☆
Recherche	★★★★★	★★★☆☆	★★★☆☆	★★☆☆☆
Production	★★★★☆	★★★★★	★★★★☆	★★★☆☆
GPU support	Excellent	Excellent	Via TensorFlow	Bon
Spécialité	Recherche, NLP, CV	Production, Mobile	Prototypage rapide	Vision par ordinateur
Communauté	Très large	Large	Large	Déclinante

Quelle stratégie adopter ?

Débutant en IA :

Commencez par Keras pour comprendre les concepts, puis passez à PyTorch.

Recherche / R&D :

PyTorch est le standard. La majorité des papiers et modèles pré-entraînés sont en PyTorch.

Production / Déploiement :

TensorFlow Serving + TF Lite pour le mobile. Mais PyTorch rattrape avec TorchServe.

III.C — Validation de l'environnement complet

Section III

Après avoir tout installé, il est essentiel de vérifier que l'ensemble fonctionne. Voici le script de test final :

```
test_environnement.py
import torch
import sys

# Test 1 : Version Python
print(f"Python : {sys.version}")

# Test 2 : Version PyTorch
print(f"PyTorch : {torch.__version__}")

# Test 3 : CUDA disponible ?
if torch.cuda.is_available():
    print(f"GPU détecté : {torch.cuda.get_device_name(0)}")
    print(f"VRAM : {torch.cuda.get_device_properties(0).total_mem / 1e9:.1f} Go")
    print("CUDA fonctionne correctement !")
else:
    print("ERREUR : GPU non détecté. Vérifiez vos drivers.")

# Test 4 : Calcul GPU rapide
if torch.cuda.is_available():
    x = torch.randn(1000, 1000, device='cuda')
    y = torch.matmul(x, x)
    print(f"Calcul GPU réussi ! Résultat : tenseur {y.shape}")
```



Si tout affiche « True » et le nom de votre GPU → votre environnement est prêt pour l'IA !

IV

Questions de compréhension

Testez vos connaissances sur les environnements matériels et logiciels



IV – Quiz de compréhension

Quiz

- 1 Quelle est la différence entre un système d'exploitation et un pilote (driver) ?
- 2 Pourquoi Linux est-il préféré à Windows pour les serveurs d'IA en production ?
- 3 Quelle est la chaîne logicielle à installer pour utiliser un GPU NVIDIA en Deep Learning ? Dans quel ordre ?
- 4 À quoi sert CUDA ? Que se passe-t-il si on ne l'installe pas ?
- 5 Pourquoi la configuration réseau est-elle importante dans un projet d'IA ?
- 6 Quelle commande permet de vérifier que le GPU est bien détecté par le système ?
- 7 Quelle est la différence entre PyTorch et Keras ? Lequel est recommandé pour un débutant ?
- 8 Comment vérifier en Python que PyTorch utilise bien le GPU ?

Prenez le temps de répondre avant de passer à la slide suivante !

IV — Réponses aux questions (1-4)

Quiz

1 OS vs Driver ?

L'OS gère l'ensemble du système (CPU, RAM, processus, fichiers). Le driver est un petit programme spécifique qui permet à l'OS de communiquer avec un composant précis (ex : GPU NVIDIA). L'OS est le chef d'orchestre ; le driver est la partition pour un instrument.

2 Pourquoi Linux en production ?

Linux est gratuit, ultra-stable (serveurs tournent des mois sans redémarrer), offre un contrôle total sur le système, consomme moins de ressources (pas d'interface graphique sur un serveur), et est le seul OS nativement supporté par les outils de cluster (SLURM, Kubernetes, Docker en production).

3 Chaîne GPU ?

L'ordre est : 1) Driver NVIDIA (communication OS ↔ GPU), 2) CUDA Toolkit (calcul parallèle sur GPU), 3) cuDNN (optimisations Deep Learning). Chaque couche dépend de la précédente — sauter une étape = GPU inutilisable pour l'IA.

4 Rôle de CUDA ?

CUDA permet d'utiliser les milliers de cœurs du GPU pour du calcul parallèle. Sans CUDA, le GPU ne sert qu'à l'affichage vidéo. PyTorch et TensorFlow utilisent CUDA en coulisses pour accélérer les calculs matriciels de l'entraînement.

IV — Réponses aux questions (5-8)

Quiz

5

Réseau en IA ?

Le réseau permet : 1) de télécharger des datasets volumineux (100+ Go), 2) d'accéder à des serveurs distants via SSH pour entraîner sur GPU, 3) de faire de l'entraînement distribué sur plusieurs machines. Un réseau mal configuré ralentit ou bloque tout le workflow.

6

Commande GPU ?

La commande `nvidia-smi` affiche : le nom du GPU, la version du driver, l'utilisation de la VRAM, la température et les processus en cours. C'est l'outil de diagnostic n°1 pour vérifier que le GPU est détecté et fonctionne.

7

PyTorch vs Keras ?

PyTorch offre plus de flexibilité et de contrôle (graphe dynamique, débogage facile). Keras est plus simple d'utilisation (API haut niveau, `model.fit()`). Pour un débutant, Keras permet de comprendre les concepts rapidement. PyTorch est recommandé ensuite pour la recherche.

8

Vérifier GPU en Python ?

La commande `torch.cuda.is_available()` retourne `True` si PyTorch détecte le GPU via CUDA. `torch.cuda.get_device_name(0)` donne le nom du GPU. Si ça retourne `False`, les drivers, CUDA ou la version de PyTorch ne sont pas correctement configurés.

CONCLUSION



L'OS est la fondation : Linux (Ubuntu) est le standard de l'IA professionnelle. Windows + WSL2 est une alternative viable pour le développement local.



Les pilotes GPU sont critiques : sans la chaîne Driver → CUDA → cuDNN, un GPU à 2000\$ est inutile pour l'IA.



Le réseau est souvent négligé mais essentiel : téléchargement de données, accès SSH, entraînement distribué.



Python est le langage obligatoire. PyTorch et TensorFlow/Keras sont les deux frameworks dominants.



Toujours valider l'environnement avec `nvidia-smi` et `torch.cuda.is_available()` avant de commencer un projet.

Prochain module : Utiliser les technologies de virtualisation dans le contexte de l'IA (Docker, Cloud, Kubernetes)

Terme	Définition
OS	Operating System. Logiciel fondamental gérant le matériel et permettant l'exécution des programmes.
Driver	Pilote. Logiciel permettant la communication entre l'OS et un composant matériel spécifique.
CUDA	Compute Unified Device Architecture. Plateforme NVIDIA de calcul parallèle sur GPU.
cuDNN	CUDA Deep Neural Network Library. Bibliothèque optimisée pour les opérations de réseaux de neurones.
SSH	Secure Shell. Protocole de connexion à distance sécurisée vers une machine Linux.
IDE	Integrated Development Environment. Logiciel regroupant tous les outils de développement.
Framework ML	Boîte à outils logicielle pré-construite pour créer et entraîner des modèles d'IA.
WSL2	Windows Subsystem for Linux 2. Permet d'exécuter Linux dans Windows avec accès GPU.
LTS	Long Term Support. Version d'OS recevant des mises à jour de sécurité pendant 5 ans.
venv	Virtual Environment. Environnement Python isolé pour éviter les conflits de bibliothèques.